

论文模板 08 · 批处理架构设计

配套素材见 [00-项目素材库.md](#)。本主题**出现频率中等**，但因多数考生缺乏真实批处理经验，竞争压力小，是**稳妥的备选题**。

一、真题常见问法

变体 A（批处理整体）

请论述你参与的某系统中批处理（离线处理）部分的设计，包括作业调度、容错重跑、性能优化等方面。

变体 B（数据导入导出）

请论述你如何设计系统的数据导入/导出功能，如何保证大批量数据处理的正确性与效率。

变体 C（作业可靠性）

请论述你如何保障定时作业的可靠性，如何处理作业失败与幂等性问题。

判断：BASE HOME 在三个变体上都有真实素材——10 个 Spring Batch Job、双数据源读写分离、幂等控制、专用重试服务、Prometheus 监控、CSV 生成与 S3 上传。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台中批处理架构的设计与实现。

该平台的批处理承担成本文件生成、自动邮件生成与发送、外部预约数据导入、主数据联动、ToDo 状态更新等核心离线业务，共 10 个 Spring Batch 作业。

本人担任架构师，负责批处理架构设计、作业可靠性保障与性能优化。

针对【批量作业与在线业务争抢数据库资源】、【消息推送重复发送风险】、【作业失败后的可恢复性】三大挑战，

本文从【读写分离双数据源】、【幂等与专用重试】、【参数化可重跑设计】三个维度进行设计：

为批处理配置独立的读写数据源并按包路径隔离 Mapper，

以幂等时间窗与独立重试服务杜绝重复推送，

将基准日期等关键参数显式参数化以保证结果可复现。

系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段 • 项目背景与批处理业务 (约 400 字)

批处理论文开头要列清「有哪些作业」，体现真实经历。

BASE HOME 的批处理承载日终与定时的离线业务，基于 Spring Batch 3.5.7 实现，共 10 个作业：

作业	业务含义
ActivityCostFileCreateJob	活动原价成本文件生成
AirCostFileCreateJob	机票成本文件生成
HotelCostFileCreateJob	酒店成本文件生成
AutomaticEmailCreateJob	自动邮件生成
MailSendJob	邮件发送
ExternalReservationImportJob	外部预约数据导入
ImportMdmJob	集团 MDM 主数据联动
GenericQueryExecutionJob	通用查询执行（元数据驱动）
TodoStatusUpdateJob	MyPage ToDo 状态更新
NoopJob	测试用空作业

这些作业的业务特征差异很大：成本文件生成是**计算密集型**（需汇总大量费用明细），邮件发送是**IO 密集型**（依赖外部网关），数据导入则是**混合型**。批处理模块设置为 `web-application-type: none`，不占用 Web 端口，与在线业务在进程级完全隔离。

第 2 段 • 架构设计 (约 800 字, 本文核心)

第一层：资源隔离——读写分离双数据源

批处理最大的架构风险不是作业本身失败，而是**批量作业拖垮在线业务**。成本文件生成需要扫描大量费用明细，外部预约导入需要回写大量记录，若与在线接口共享数据库连接池，夜间批次高峰会直接推高在线接口延迟。

我的方案是配置**双数据源**： - `ReaderDataSourceConfig` (`spring.datasource.reader`) —— 只读账号，承担大量查询 - `WriterDataSourceConfig` (`spring.datasource.writer` , 标 `@Primary`) —— 写账号，承担结果回写

通过**包路径隔离 Mapper** (`repository/reader/` 与 `repository/writer/`) , 使读写走不同数据源，物理层面互不干扰。配合的优化还包括： - 事务隔离级别设为 `TRANSACTION_READ_COMMITTED` , 避免长事务期间持有大量间隙锁影响在线写入 - `mybatis.defaultFetchSize = 200` 控制批量游标，防止一次性载入海量结果集导致 JVM 内存溢出——这是批处理最典型的故障模式

权衡说明：双数据源增加了配置复杂度，且跨数据源无法使用单一事务（需业务上避免跨源事务），但换来的在线业务稳定性收益远大于此成本。

第二层：作业组织与复用

10 个作业统一继承 `job/base/ManagedBaseJob` 基类，把作业的公共能力（参数解析、日志、异常处理）收敛到基类，各作业只关注自身业务逻辑。业务逻辑下沉到 `service/` 层的对应服务

(`MailSendService` 、 `ImportMdmService` 、 `FileCreateService` 等) , 使作业类只承担编排职责，业务逻辑可被测试与复用。作业参数通过枚举统一定义 (`JobParameterKey`) : `acquisitionRange` (取得范围)、 `executionBaseDate` (执行基准日)、 `todoDistinction` 、 `emailType` 、 `importTargetType` 。

第三层：作业的可靠性设计

- **幂等控制**： `app.batch.idempotency-duration-minutes = 10` 提供时间窗去重，防止重复执行造成重复推送
- **专用重试服务**： `MailSendRetryService` 独立承担邮件重试，与业务解耦，重试策略可独立调整
- **事务粒度控制**： `service/transaction/*TransactionalService` 把事务边界限制在必要范围，避免大事务回滚造成连带重放
- **可观测性**：通过 **Prometheus PushGateway** 上报作业指标 (`shutdown-operation: post` 确保短生命周期进程在退出前完成指标推送) , 使定时作业的运行状态可被监控系统采集与告警

第四层：元数据驱动的通用查询

`GenericQueryExecutionJob` 配合 `MGenericQuery*` 系列表，把查询逻辑外置为元数据配置，新增一类查询只需配置数据而无需改代码。这一设计显著提升了批处理的**可修改性**——业务侧新增报表类查询不再需要开发排期。

第 3 段 • 关键技术实现 (约 700 字)

要素一：CSV 文件生成与上传

三类成本文件 (ACT/AIR/HTL) 使用 **SuperCSV 注解库**实现，通过注解将实体字段直接映射为 CSV 列，导出列定义集中在实体上 (`ActivityCostDownloadCsv` 、 `AirCostDownloadCsv` 、 `HotelCostDownloadCsv`) ，配合 `CsvDownloader` 接口与 `FileUploadService` 落 S3。设计价值：新增导出列时只需修改实体注解，不会波及导出逻辑代码——这是「限制变更影响范围」的可修改性战术。

要素二：外部数据导入

外部预约数据导入 (`ExternalReservationImportJob`) 通过 `RecvImportType` 枚举区分导入类型，经 `ExternalReservationImportService` 处理入库。集团主数据联动 (`ImportMdmJob`) 通过 `api/mdm/` 封装的认证与查询参数访问 MDM API，本地落为 26 张 `M*` 主数据表。

要素三：消息通知联动

批处理完成后需推送通知，通过 `SnsService` 发布到 SNS 主题 (`app.services.aws.sns`) ，由下游 Lambda 消费。这种「批处理产出 → SNS → Lambda 消费」的模式，把批处理与通知渠道解耦：新增通知渠道无需修改批处理代码。

要素四：作业参数的显式化

我把 `executionBaseDate` (执行基准日)、`acquisitionRange` (取得范围) 等关键参数定义在 `JobParameterKey` 枚举中并显式传入，而非在代码中直接取系统当前时间。这一设计带来两个收益：① **可重跑**——补数时指定历史日期即可重新生成，结果与当日运行一致；② **可测试**——测试可固定参数，使结果可预期。这对跨国系统尤其重要：服务日本本部 (源库 JST) 而容器运行于 UTC，若依赖系统时钟取「今天」，跨时区下极易取到错误的日期边界。

第 4 段 • 难点与解决 (约 700 字)

批处理主题最佳组合：

优先级	难点	适配理由
首选	⑦ 批量推送的幂等与重试	主题核心
次选	④ MySQL 读写分离双数据源	直接对应「资源隔离」
三选	⑧ 跨国多时区一致性	本主题的杀手级素材，直接对应「作业正确性」
加分	⑩ MDM 主数据同步	对应「数据导入」

难点示例 (⑧ 多时区，批处理视角)：

问题：系统服务日本本部，源数据库运行于 JST，而批处理容器环境为 UTC。这个差异在「取当天数据生成成本文件」这类作业上会直接造成错误——容器认为的「今天」在 JST 下可能还是「昨天」，反之亦然。后果是成本文件漏当天的数据或重复包含前一天的数据，而这类错误不会抛出异常，只会静默地产生错误结果。

方案：我的处理分两处。第一处是**存储口径统一**：数据库中的时间统一以 UTC 存储，展示与业务判断时按 JST 转换，转换动作集中到 `FormatUtils` 工具类，禁止业务代码中散落直接取当前时间的调用。第二处是**批处理参数化**：把 `executionBaseDate` 作为 `JobParameter` 显式传入而不依赖系统时钟，使作业的日期边界由调用方决定。这带来的额外收益是**可重放性**——指定同一基准日重跑，结果与当日运行完全一致，补数场景可靠。

效果：◆ 时区相关缺陷数量下降情况；作业结果可复现。

第 5 段 · 总结与不足 (约 200 字)

目前 10 个批处理作业已稳定运行【◆ 时长】，成本文件与邮件在【◆ 时限】内稳定完成，未发生重复推送事故。

不足之处：

一是幂等控制基于时间窗，理论上在窗口边界仍有极小概率的重复；

二是作业间的依赖关系（如邮件生成必须先于邮件发送）目前依靠调度时序保障，尚未引入显式的依赖编排。

后续计划引入基于持久化状态位的幂等机制与作业依赖编排。

通过本项目我认识到，批处理架构的关键不在于“批”的效率，而在于“失败之后能不能干净地重来”——

幂等与可重跑，比处理速度更重要。

四、本项目填充点速查

论文要素	直接可写的内容
框架	Spring Batch 3.5.7 (Spring Boot 3.5.7)
作业数量	10 个 (含 1 个测试用 <code>NoopJob</code>)
作业基类	<code>job/base/ManagedBaseJob</code>
成本文件作业	<code>ActivityCostFileCreateJob</code> / <code>AirCostFileCreateJob</code> / <code>HotelCostFileCreateJob</code>
邮件作业	<code>AutomaticEmailCreateJob</code> (生成) / <code>MailSendJob</code> (发送)
导入作业	<code>ExternalReservationImportJob</code> / <code>ImportMdmJob</code>
通用查询	<code>GenericQueryExecutionJob</code> + <code>MGenericQuery*</code> 元数据表
ToDo 作业	<code>TodoStatusUpdateJob</code>

论文要素	直接可写的内容
读写分离	<code>ReaderDataSourceConfig</code> / <code>WriterDataSourceConfig</code> (<code>@Primary</code>)
Mapper 隔离	<code>repository/reader/</code> 与 <code>repository/writer/</code>
事务隔离	<code>TRANSACTION_READ_COMMITTED</code> (Hikari 配置)
批量游标	<code>mybatis.defaultFetchSize = 200</code>
幂等	<code>app.batch.idempotency-duration-minutes = 10</code>
重试服务	<code>service/retry/MailSendRetryService</code>
事务服务	<code>service/transaction/*TransactionalService</code>
作业参数	<code>JobParameterKey</code> : <code>acquisitionRange</code> / <code>executionBaseDate</code> / <code>todoDistinction</code> / <code>emailType</code> / <code>importTargetType</code>
命令类型	<code>CommandType</code> : <code>DELETE</code> / <code>INSERT</code> / <code>UPDATE</code> / <code>DDL</code> / <code>NOT_FOUND</code>
CSV	SuperCSV 注解库; <code>CsvDownloader</code> 接口; <code>FileUploadService</code> 上传 S3
输出类型	<code>CostFile</code> (ACT/AIR/HTL) 、 <code>EmailType</code> 、 <code>SenderName</code> 、 <code>SubjectHeader</code>
消息推送	<code>SnsService</code> → SNS 主题; <code>MailSendService</code>
监控	Prometheus PushGateway (<code>shutdown-operation: post</code>)
运行模式	<code>spring.main.web-application-type: none</code> (无 Web 容器)
邮件发送配置	<code>sender = log</code> (测试用日志代替实际发送) 、 <code>sleep-after-send = 0</code>
主数据联动	<code>api/mdm/</code> (认证 + 查询参数) , <code>MdmContent</code> 枚举
测试	TestContainers + MySQL 镜像; <code>NoopJob</code> / <code>NoopService</code> 便于集成测试

五、可用难点素材（按适配度排序）

- ⑦ 批量推送的幂等与重试 ★★ ★ —— 本主题首选
- ⑧ 跨国多时区一致性 ★★ ★ —— 本主题的差异化杀手素材
- ④ MySQL 读写分离双数据源 ★★ ★ —— 资源隔离
- ⑩ MDM 主数据统一 ★★ —— 数据导入
- ② protobuf 版本冲突 ★★ —— 依赖治理（可放"构建可靠性"）

六、踩坑提醒

坑	后果	正确做法
只讲「用了 Spring Batch」	无深度	写清作业划分、参数化、幂等、重试、监控
不提幂等	关键失分	批处理论文 必写幂等 ，这是可靠性核心
不提作业失败处理	漏考点	写「失败可观测 (Prometheus) + 可重跑 (参数化)」
只说「性能优化用了多线程」	高风险	本项目未见分区 (Partitioning) 并行证据，不要编造
编造 Quartz/XXL-Job	高风险	本项目未见独立调度框架，不要写；调度方式如实描述即可
忽略与在线业务的隔离	漏考点	读写分离是本项目真实亮点，务必写
忘记写业务含义	显得空洞	每个作业都要说明它支撑什么业务 (成本文件、邮件、ToDo...)

⚠ 技术真实性提醒： 本项目**未见** Spring Batch 的分区 (Partitioning) 或多线程 Step 的证据，也**未见** XXL-Job / Quartz 等外部调度框架。论文中不要编造「我们使用了分片并行处理」。真实的性能手段是：**读写分离、游标控制、事务隔离级别、元数据驱动查询**——这四点已经足够撑起「性能优化」这一段。

配套： [03- 数据架构与混合存储.md](#) · [04- 高并发与高可用架构.md](#) · [02- 云原生与FaaS架构.md](#)